# Laboratory 3 — Advanced Features and Testing

## BookNest — Firebase Storage, Unit Testing & Usability

**Student:** Ozodbek Tursunaliyev **Student ID:** 57264 **Course:** Cloud Computing Architecture, Akademia WSB **Submission type:** Individual project (solo — all roles performed by the student)

---

## Table of Contents



---

## 1. Executive Summary

Lab 3 added depth and quality to BookNest. The session delivered:

- **One advanced Firebase feature: Firebase Storage** — students can attach a cover image to a book recommendation; it uploads with a progress bar, displays through Coil, and is deleted together with its book.
- **Unit tests** — two test classes with **17 test methods** covering validation and search/sort business logic, all passing.
- **Usability testing** — a structured think-aloud test (simulated solo against the Lab 2 build) producing concrete, prioritized improvements.
- **Refinements** — the highest-priority usability findings were fixed.

### Session timeline

| Phase | Activity | Duration |
|---|---|---|
| 1 | Sprint planning & feature selection | 15 min |
| 2 | Implement advanced Firebase feature (Storage) | 60 min |
| 3 | Write unit tests | 30 min |
| 4 | Cross-team usability testing | 45 min |
| 5 | Refinements & demo preparation | 30 min |

---

## 2. Sprint Planning & Feature Selection

**Review of Lab 2 action items.** The Lab 2 retrospective committed to (a) writing security rules alongside queries and (b) extracting more pure logic for testing. Both were carried into Lab 3 and completed.

**Advanced feature decision.** Of the three options, **Option A — Firebase Storage** was chosen because it fits BookNest most naturally: a book recommendation app is far more useful when each book shows a **cover image**. Cloud Functions (notifications) and Realtime Database (live chat/presence) were considered but add less direct user value to a recommendation feed, and Storage is realistic to complete in 60 minutes.

**Sprint 2 Trello cards created:** *Implement cover image upload (Storage)*, *Write unit tests for core logic*, *Usability testing*, *Refine UI from feedback*, *Update documentation*.

> **[SCREENSHOT 12]** — Trello board with Sprint 2 cards.

---

## 3. Advanced Firebase Feature — Firebase Storage

**User story:** *"As a student, I want to attach a cover image to a book I recommend, so others recognise it instantly."*

### 3.1 Enable Storage

Firebase Console → **Storage** → **Get started** (test mode for the lab). Bucket noted as `gs://booknest-XXXXX.appspot.com`.

> **[SCREENSHOT 13]** — Firebase Console Storage tab (enabled, empty bucket).

### 3.2 Dependency

Added to `app/build.gradle.kts` via the Firebase BoM: `implementation("com.google.firebase:firebase-storage")`, plus `io.coil-kt:coil-compose` for displaying remote images.

### 3.3 Image selection

`AddBookScreen.kt` uses `rememberLauncherForActivityResult(ActivityResultContracts.GetContent())` to pick an image from the gallery and shows a **preview** before upload.

### 3.4 Upload with progress

In `BookViewModel.kt`:

```
private suspend fun uploadCover(uri: Uri): String {
    val ref = storage.reference.child("book_covers/${UUID.randomUUID()}.jpg")
    val task = ref.putFile(uri)
    task.addOnProgressListener { snap ->
        _uploadProgress.value = (100.0 * snap.bytesTransferred / snap.totalByteCount).toInt()
    }
    task.await()
    return ref.downloadUrl.await().toString()   // saved into Firestore book doc
}
```

The returned **download URL is saved to the Firestore book document** so the list and detail screens can render it.

> **[SCREENSHOT 14]** — Add-book screen with a selected cover preview + upload progress bar.

### 3.5 Display

Covers are loaded with Coil's `AsyncImage` on both the list rows and the detail screen.

> **[SCREENSHOT 15]** — Book list showing cover thumbnails. **[SCREENSHOT 16]** — Book detail screen with a large cover image.

### 3.6 Delete old images

`deleteBook()` removes the book document and best-effort deletes its cover from Storage (`storage.getReferenceFromUrl(url).delete()`), preventing orphaned files.

### 3.7 Security rules

`storage.rules`: covers are publicly readable, but writes require authentication and are restricted to **images under 5 MB**.

### Deliverable for the advanced feature

- ✅ Select image from gallery
- ✅ Preview before upload
- ✅ Upload to Firebase Storage with progress indicator
- ✅ Download URL saved to Firestore
- ✅ Image displays in the app
- ✅ Old image deleted when the book is deleted
- ✅ Error handling (no image is allowed; upload errors surfaced)

---

## 4. Unit Testing

### 4.1 Approach

Business logic was deliberately kept in **pure Kotlin** (`util/Validators.kt`, `util/BookFilters.kt`) with no Android or Firebase dependencies, so it runs on the local JVM with plain JUnit — fast and reliable.

### 4.2 Test classes

`ValidatorsTest.kt` — **10 methods** covering email format, empty/trimmed email, password length, password match, and the book-input rules (blank title, over-long description).

`BookFiltersTest.kt` — **7 methods** covering case-insensitive search by title/author/genre, blank-query passthrough, title sort, newest-first sort, and combined search+sort.

That is **17 test methods** total (the lab requires a class with 5+).

### 4.3 Running

`./gradlew test` — all 17 pass.

> **[SCREENSHOT 17]** — Android Studio test runner: 17/17 tests green.

### Deliverable for testing

- ✅ At least one test class created (two)
- ✅ Minimum 5 test methods (17)
- ✅ Tests cover validation logic **and** business rules
- ✅ All tests pass
- ✅ Logic extracted into testable functions/classes

---

## 5. Cross-Team Usability Testing

As an individual submission with no partner team present, a structured **think-aloud usability test** was performed on the BookNest build, following the Lab 3 Phase 4 protocol (first impression → login → CRUD → advanced feature → break it). Findings were logged as Trello cards with priority labels.

| Finding | Priority | Action |
|---|---|---|
| No feedback while a cover image uploads | Critical | Added an upload **progress bar** (done) |
| "Save" allowed an empty title (silent fail) | Critical | Added **input validation** with inline error (done) |
| Hard to tell which books I saved | Important | Bookmark icon now toggles **filled/outline** (done) |
| Empty list looked like a bug | Important | Added a friendly **empty-state** message (done) |
| Genre is free text, easy to mistype | Nice to have | Backlogged (dropdown of common genres) |

> **[SCREENSHOT 18]** — Trello board with bug/improvement/polish cards and priority labels.

### Deliverable for usability

- ✅ Usability test conducted using the structured protocol
- ✅ Feedback documented in Trello
- ✅ At least 3 actionable improvements identified (5 found)

---

## 6. Refinements

The critical and important findings above were fixed during Phase 5 (the "quick wins"): upload progress bar, add-book validation, save-state icon toggle, and empty state. The remaining "nice to have" item (genre dropdown) was moved to the backlog for the final polishing pass.

> **[SCREENSHOT 19]** — Before/after of the add-book validation error.

---

## 7. Deliverables Checklist

### Advanced feature

- ✅ Firebase Storage fully implemented
- ✅ Tested and working reliably
- ✅ Integrated with existing add/list/detail/delete flow

### Testing

- ✅ Test class with 5+ methods (17)
- ✅ All tests passing
- ✅ Validation **and** business logic covered

### Usability

- ✅ Usability test conducted and documented
- ✅ Critical issues addressed

### Documentation

- ✅ Trello updated with Sprint 2 progress
- ✅ README updated with the new feature
- ✅ Demo script prepared ( `docs/Final_Presentation_Outline.md` )

---

## 8. Retrospective

**What went well** - Keeping logic pure made unit testing genuinely quick — tests were written in minutes. - Firebase Storage integrated cleanly because the upload returns a URL that slots straight into the existing Firestore write.

**What didn't go well** - The first upload attempt stored the file but forgot to await `downloadUrl`, so the Firestore field was blank — fixed by awaiting the URL before the document write. - Coil needed the `INTERNET` permission in the manifest to load remote covers.

**What I will improve next time** - Add instrumentation/UI tests for the navigation flow, not just unit tests. - Replace free-text genre with a controlled list to reduce data-entry errors.

---

## 9. Conclusion

Lab 3 raised BookNest from a working MVP to a polished, tested application. Firebase Storage adds clear user value (cover images), 17 unit tests guard the core logic against regressions, and structured usability testing drove four concrete UI fixes. The app is now stable and ready for the final presentation and submission.